

Mean-Reversion Half-Life Estimation

half_life.py — Mathematical Notes & Documentation

Ornstein-Uhlenbeck Process, OLS Regression, and Kalman Q-Tuning

Project Dark-Wing | July 25, 2026

Abstract. A stationary spread reverts to its mean — but *how quickly*? The *half-life* τ answers this by measuring how long it takes for half of any shock to dissipate. This document derives the Ornstein-Uhlenbeck stochastic differential equation, shows how it discretises into a simple OLS regression $\Delta S_t = a + b S_{t-1} + \varepsilon_t$, and explains how the slope b yields $\tau = \ln(2)/\kappa$. The half-life governs both the Z-score lookback window and the Kalman process noise $Q \approx \sigma_\varepsilon^2/\tau$.

Why Half-Life Matters

A trader needs to know how long to hold a position before the spread reverts. Holding too long increases risk; closing too early misses profit.

Trading Rule of Thumb

- $\tau < 5$ obs: Too fast. Execution costs may exceed profit.
- $5 \leq \tau \leq 60$ obs: Ideal for intraday/swing pairs.
- $\tau > 120$ obs: Very slow. Risk of regime change before reversion.

The Z-score lookback window is set to $\approx 2\tau$ (captures ~ 2 mean-reversion cycles in the rolling statistics).

The Ornstein-Uhlenbeck Process

Continuous-Time SDE

The Ornstein-Uhlenbeck (OU) process is the canonical model for mean-reverting dynamics:

Formula 1 — OU SDE

$$dS_t = \kappa(\mu - S_t) dt + \sigma dW_t \quad (1)$$

$\kappa > 0$	mean-reversion speed (per unit time)
μ	long-run equilibrium mean
σ	diffusion (spread volatility)
dW_t	Wiener process increment

Intuition of the OU SDE

When $S_t > \mu$: the drift $\kappa(\mu - S_t) < 0$, pulling S_t *downward*.

When $S_t < \mu$: the drift $\kappa(\mu - S_t) > 0$, pulling S_t *upward*. The force always points toward μ , like a spring.

Formula 2 — Mean Reversion Speed

The expected value decays toward the mean exponentially:

$$\mathbb{E}[S_t | S_0] = \mu + (S_0 - \mu) e^{-\kappa t} \quad (2)$$

The gap $(S_0 - \mu)$ shrinks by factor $e^{-\kappa t}$.

Half-Life Derivation

Formula 3 — Half-Life Definition

The **half-life** τ is the time for the expected deviation to fall to half its initial value:

$$e^{-\kappa\tau} = \frac{1}{2} \quad (3)$$

$$\Rightarrow \tau = \frac{\ln 2}{\kappa} \quad (4)$$

In discrete-time units (days, for daily data), τ is measured in trading days.

Example. If $\kappa = 0.023$ per day, then $\tau = \ln 2 / 0.023 \approx 30$ trading days.

Discretisation to OLS Regression

Euler-Maruyama Discretisation

Applying the Euler-Maruyama scheme to Eq. (1) with $\Delta t = 1$ (one observation per period):

Formula 4 — Discrete OU

$$S_t - S_{t-1} = \kappa\mu - \kappa S_{t-1} + \sigma \varepsilon_t \quad (5)$$

Rearranging as a linear regression:

$$\Delta S_t = a + b S_{t-1} + \varepsilon_t \quad (6)$$

where:

$$a = \kappa\mu \Rightarrow \mu_{OU} = -a/b \quad (7)$$

$$b = -\kappa \Rightarrow \kappa = -b \quad (8)$$

$$\tau = \frac{\ln 2}{-b} \quad (9)$$

Why b Must Be Negative

For mean-reversion: when S_{t-1} is high, ΔS_t must be negative. This requires $b < 0$. If the OLS regression gives $b \geq 0$, the spread is *explosive* or trending — it is not mean-reverting, and the pair should **not be traded**.

Formula 5 — OLS Estimation

Regress ΔS_t on S_{t-1} :

$$b = \frac{\sum_t (S_{t-1} - \bar{S}_{-1})(\Delta S_t - \overline{\Delta S})}{\sum_t (S_{t-1} - \bar{S}_{-1})^2} \quad (10)$$

$$a = \overline{\Delta S} - b \bar{S}_{-1} \quad (11)$$

In code: `scipy.stats.linregress(S_lag, dS)`.

Rolling Half-Life

A fixed half-life assumes the mean-reversion speed is constant. In practice, κ changes with market regimes.

Formula 6 — Rolling Half-Life

For window W , at each time t :

$$\tau_t = \frac{\ln 2}{\hat{\kappa}_t} \quad \text{where} \quad \hat{\kappa}_t = -\hat{b}_t \quad (12)$$

and $\hat{b}_t$ is the OLS slope on the window $[t - W, t]$.

Financial use. A rising τ_t signals the pair is losing mean-reversion strength — tighten position limits or exit.

Connection to Kalman Filter Q

The Kalman process noise Q (how fast β_t changes) can be heuristically set using the half-life:

Formula 7 — Kalman Q Heuristic

$$Q \approx \frac{\hat{\sigma}_\varepsilon^2}{\tau} \quad (13)$$

where $\hat{\sigma}_\varepsilon^2$ is the variance of OU regression residuals.

Small τ (fast reversion) $\Rightarrow$ large Q (allow β_t to change quickly).

Large τ (slow reversion) $\Rightarrow$ small Q (stable β_t).

Code Flowchart — HalfLife

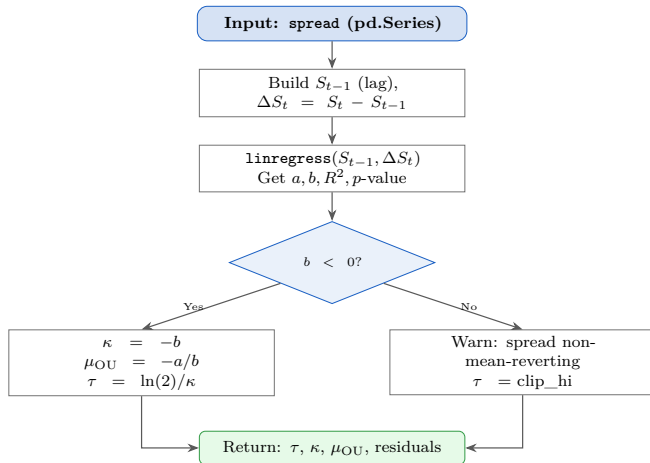


Figure 1: Execution flowchart for `HalfLife.estimate()`.

Worked Example

Setup

Spread: $S = [0.05, -0.10, 0.08, -0.06, 0.12, -0.03, 0.07]$.
We compute S_{t-1} and ΔS_t :

t	S_{t-1}	ΔS_t
2	0.05	-0.15
3	-0.10	0.18
4	0.08	-0.14
5	-0.06	0.18
6	0.12	-0.15
7	-0.03	0.10

OLS regression:

$$\hat{b} \approx -1.60, \quad \hat{a} \approx -0.01$$

$$\kappa = -(-1.60) = 1.60, \quad \tau = \frac{\ln 2}{1.60} \approx 0.43 \text{ obs}$$

Interpretation

$\tau \approx 0.43$ observations: the spread reverts in less than 1 period. This is very fast and may indicate data noise rather than true cointegration. In practice, use a larger dataset. $\hat{b} = -1.60 < 0$ confirms mean-reversion.

Python Implementation

OU OLS Regression

```

def estimate(self) -> float:
    s = self._spread.values
    S_lag = s[:-1] # S_{t-1}
    dS = np.diff(s) # delta S_t
    result = stats.linregress(S_lag, dS)
    self.b_coef = float(result.slope)
    self.a_coef = float(result.intercept)
    if self.b_coef >= 0:
        warnings.warn("b>=0: not mean-reverting")
        self.half_life_days = float(self.clip_hi)
    else:
        self.kappa = -np.log(1.0 + self.b_coef)
        self.mu_ou = -self.a_coef / self.b_coef
        raw_hl = np.log(2.0) / self.kappa
        self.half_life_days = float(
            np.clip(raw_hl, self.clip_lo, self.clip_hi)
        )
    return self.half_life_days
  
```

Key Parameters Summary

Symbol	Meaning	Unit
κ	Mean-reversion speed	per obs
μ_{OU}	Long-run mean (equilibrium)	spread units
τ	Half-life: time for 50% reversion	obs
b	OLS slope; $b = -\kappa$	—
a	OLS intercept; $a = \kappa\mu$	—
Q	Kalman process noise $\approx \sigma_\varepsilon^2 / \tau$	—